

Python Tip: the `in` Operator

Need to check if something is contained in a list?

Want to see if a string contains a substring?

Use the `in` operator for a clean, readable one-liner!



Checking Lists

Use `in` to check if a value exists in a list.

```
fruits = ["apple", "banana", "cherry"]
```

```
# Don't do this
```

```
found = False
```

```
for fruit in fruits:
```

```
    if fruit == "banana":
```

```
        found = True
```

```
print(found)
```

```
# Do this instead
```

```
print("banana" in fruits)
```

```
print("grape" in fruits)
```

```
True
```

```
True
```

```
False
```

Checking Strings

`in` also works on strings to check for substrings.

```
message = "Hello, world!"  
  
print("world" in message)  
print("python" in message)
```

True
False

Checking Dictionaries

With dictionaries, `in` checks the **keys** by default.

```
scores = {"alice": 95, "bob": 87, "carol": 92}
# Check keys
print("alice" in scores)
# Check values
print(95 in scores.values())
# Check pairs
print(("bob", 87) in scores.items())
```

True

True

True

Using `not in`

Flip the check with `not in` to test for absence.

```
banned = ["spam", "ads", "clickbait"]  
  
username = "spam"  
if username not in banned:  
    print(f"Welcome, {username}!")  
else:  
    print(f"{username} is banned.")
```

```
spam is banned.
```

Wrap-Up

Now you can use `in` and `not in` to check membership in lists, strings, and dictionaries cleanly and concisely!

Follow me for more tips.

Shep Bryan IV